

Rezumat Curs 4 — Rețele Neuronale Artificiale (RNA / ANN)

Surse: 04_ML_ANN.pdf + 4_1 → 4_9 fișiere .md UBB Informatică Anul 2 — Inteligență Artificială

1. Ce sunt Rețelele Neuronale Artificiale?

1.1 Definiție

Rețelele neuronale artificiale (RNA) sunt **modele computaționale inspirate din creierul uman**. Sunt formate dintr-o mulțime de noduri (neuroni) dispuse pe straturi, conectate prin legături ponderate.

1.2 De ce RNA?

Multe sarcini sunt ușor de realizat de oameni dar greu de codificat algoritmic: recunoașterea fețelor, conducerea mașinilor, recunoașterea vocii. RNA pot **învăța din date** fără reguli explicite.

1.3 Inspirația biologică

Rețea biologică	Rețea artificială
Soma (corpul celulei)	Nod (neuron)
Dendrite	Intrări
Axon	Ieșire
Sinapsă	Conexiune ponderată
Activare neuronală	Funcție de activare

Creierul uman: ~10 miliarde neuroni, ~5000 conexiuni/neuron, învață prin adaptarea conexiunilor.

2. Structura unei RNA

2.1 Straturile

Strat	Rol	Nr. de noduri
Strat de intrare	Citește datele de intrare	m (nr. de atribute)
Strat(uri) ascuns(e)	Efectuează calcule	Ales de proiectant
Strat de ieșire	Furnizează rezultatele	R (nr. de ieșiri / clase)

2.2 Datele de antrenament

Forma: (x^d, t^d) cu $d = 1, 2, \dots, n$

Unde:

- $x^d = (x_1^d, x_2^d, \dots, x_m^d) \in \mathbb{R}^m$ — vectorul de attribute (intrări)
- $t^d = (t_1^d, t_2^d, \dots, t_R^d) \in \mathbb{R}^R$ — vectorul de ieșiri dorite (etichete)
- n = numărul de exemple de antrenament
- m = numărul de attribute
- R = numărul de ieșiri

2.3 Notății pentru o RNA cu un singur strat ascuns (RNA₁)

- m neuroni pe stratul de intrare
- H neuroni pe stratul ascuns
- R neuroni pe stratul de ieșire
- w_{ih}^1 = ponderea dintre neuronul de intrare i și neuronul ascuns h ($i = 1, \dots, m; h = 1, \dots, H$)
- w_{hr}^2 = ponderea dintre neuronul ascuns h și neuronul de ieșire r ($h = 1, \dots, H; r = 1, \dots, R$)

Total ponderi (fără bias): $m \times H + H \times R$

3. Neuronul — Unitatea Primitivă de Calcul

3.1 Structura unui neuron

Un neuron primește intrări, le ponderează, le sumează, adaugă bias și aplică o funcție de activare:

$$z = \sum_{i=1}^n w_i x_i + b$$

$$y = f(z)$$

Unde:

- x_i = intrarea i a neuronului
- w_i = ponderea (weight) asociată intrării i — controlează **cât de mult contribuie** intrarea i la ieșire
- b = bias — deplasează funcția de activare (fără el, totul ar trece prin origine)
- z = suma ponderată + bias (ieșirea brută)
- f = funcția de activare
- y = ieșirea neuronului (activarea)

Exemplu: Neuron cu 2 intrări ($x_1 = 3, x_2 = 2$), ponderi ($w_1 = 0.5, w_2 = -0.1$), bias $b = 0.2$:

$$z = 0.5 \cdot 3 + (-0.1) \cdot 2 + 0.2 = 1.5 - 0.2 + 0.2 = 1.5$$

Dacă funcția de activare e sigmoid: $y = \frac{1}{1+e^{-1.5}} \approx 0.82$

3.2 Formă vectorială (pentru un strat)

Pentru un strat cu m neuroni de intrare și H neuroni de ieșire:

$$Z = X \cdot W + b$$

$$A = f(Z)$$

Unde:

- $X \in \mathbb{R}^{\text{batch} \times m}$ — matricea de intrări
- $W \in \mathbb{R}^{m \times H}$ — matricea de ponderi
- $b \in \mathbb{R}^H$ — vectorul de bias
- Z = ieșirea brută
- A = ieșirea activată

3.3 Interpretare geometrică

Componentă	Efect geometric	Exemplu
Weights (W)	Rotire și scalare a hiperplanului de decizie	Schimbă panta dreptei
Bias (b)	Translație (deplasare verticală)	Mută dreapta în sus/jos

4. Funcții de Activare

4.1 De ce avem nevoie de funcții de activare?

Fără ele, oricâte straturi am avea, RNA ar fi doar o combinație liniară (echivalentă cu un singur strat). Funcțiile de activare introduc **neliniaritate**, permițând rețelei să învețe relații complexe.

4.2 Funcțiile principale

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Domeniu: $(-\infty, +\infty) \rightarrow$ Codomeniu: $(0, 1)$
- Utilizată în regresia logistică și în stratul de ieșire pentru clasificare binară

ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z) = \begin{cases} 0 & \text{dacă } z < 0 \\ z & \text{dacă } z \geq 0 \end{cases}$$

- Continuă, monotonă
- Codomeniu: $[0, +\infty)$
- Cea mai folosită în straturile ascunse ale rețelelor adânci

Leaky ReLU (variantă)

$$f(z) = \begin{cases} a \cdot z & \text{dacă } z < 0 \\ z & \text{dacă } z \geq 0 \end{cases}$$

Unde a este un coeficient mic (ex: 0.01). Compensează problema neuronilor „morți” de la ReLU standard.

Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Domeniu: $(-\infty, +\infty) \rightarrow$ Codomeniu: $(-1, 1)$
- Simetrică față de 0

Softmax (pentru stratul de ieșire — clasificare multclasă)

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Unde:

- z_i = scorul brut (logit) pentru clasa i
- K = numărul total de clase
- Ieșire: probabilități care se însumează la 1

Exemplu: Scoruri brute $z = [2.0, 1.0, 0.1]$

$e^{2.0} \approx 7.39$, $e^{1.0} \approx 2.71$, $e^{0.1} \approx 1.11$. Total: 11.21

Probabilități: $[\frac{7.39}{11.21}, \frac{2.71}{11.21}, \frac{1.11}{11.21}] = [0.659, 0.242, 0.099]$

Clasa prezisă: $\text{argmax} = 0$ (prima clasă)

Tabel comparativ funcții de activare

Funcție	Formulă	Codomeniu	Utilizare principală
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Ieșire clasificare binară
ReLU	$\max(0, z)$	$[0, +\infty)$	Straturi ascunse
Leaky ReLU	$\max(az, z)$	$(-\infty, +\infty)$	Straturi ascunse (evită neuroni morți)
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Straturi ascunse (alternativă)
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0, 1)$, suma=1	Ieșire clasificare multclasă

4.3 Softmax vs ArgMax

Funcție	Ce face	Când se folosește
Softmax	Produce probabilități	În timpul antrenării
ArgMax	Alege clasa cu probabilitatea maximă	În timpul testării / predicției

5. Funcții de Cost (Loss Functions)

5.1 MSE — Mean Squared Error (Quadratic Cost)

$$E = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

Unde:

- $y^{(i)}$ = valoarea reală
- $\hat{y}^{(i)}$ = valoarea prezisă
- m = număr de exemple

Utilizare: Probleme de **regresie** (ieșire continuă).

5.2 Cross Entropy (Binary)

$$E = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Utilizare: **Clasificare binară** (2 clase). Penalizează puternic predicțiile confident greșite.

5.3 Cross Entropy (Multiclasă)

$$E = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log(\hat{y}_j^{(i)})$$

Unde:

- K = numărul de clase
- $y_j^{(i)}$ = eticheta corectă (one-hot: 1 pentru clasa corectă, 0 altfel)
- $\hat{y}_j^{(i)}$ = probabilitatea prezisă pentru clasa j (din softmax)

Exemplu: Etichetă reală: $y = [0, 1, 0]$ (clasa 2). Predicție: $\hat{y} = [0.2, 0.7, 0.1]$

$E = -\log(0.7) \approx 0.357$ — doar clasa corectă contribuie!

5.4 Comparație

Funcție	Potrivit pentru	Avantaj
MSE	Regresie	Simplu de înțeles
Cross Entropy	Clasificare	Gradientul rămâne mare → învățare rapidă

6. Perceptronul

6.1 Ce este?

Cel mai simplu neuron — clasificator liniar binar. Calculează suma ponderată a intrărilor și aplică un prag:

$$o = \begin{cases} 1 & \text{dacă } \sum_{i=0}^m w_i x_i > 0 \\ -1 & \text{altfel} \end{cases}$$

6.2 Limitările perceptronului

Un perceptron poate învăța **AND** și **OR**, dar **NU poate învăța XOR** (nu e liniar separabilă — nu poți trage o singură dreaptă care să separe clasele).

Soluții:

- Neuron cu un prag continuu (sigmoid) — în loc de prag discret
- Mai mulți neuroni (mai multe straturi) — rețele multi-strat (MLP)

7. Backpropagation — Algoritmul de Antrenare

7.1 Principiul

Backpropagation = Gradient Descent aplicat pe rețele cu mai multe straturi.

- **Informația utilă** se propagă **înainte** (forward) — de la intrare spre ieșire
- **Eroarea** se propagă **înapoi** (backward) — de la ieșire spre intrare

7.2 Algoritmul (pseudocod)

```
Se inițializează ponderile (aleator sau 0)
Cât timp NU este îndeplinită condiția de oprire:
    Pentru fiecare exemplu (xd, td):
        1. Forward pass — se activează fiecare neuron, se calculează
        ieșirea
        2. Se calculează eroarea la ieșire: δr = tr - or
        3. Backward pass — se propagă eroarea înapoi prin rețea
        4. Se actualizează ponderile
```

7.3 Forward Pass — Calculul ieșirii

Stratul ascuns (neuronul h):

$$o_h = \sigma \left(\sum_{i=1}^m w_{ih}^1 \cdot x_i \right) \quad \text{sau} \quad o_h = \text{sig} \left(\sum_{i=1}^m w_{ih}^1 \cdot x_i \right)$$

Stratul de ieșire (neuronul r):

$$o_r = \sigma \left(\sum_{h=1}^H w_{hr}^2 \cdot o_h \right)$$

7.4 Backward Pass — Propagarea erorii

Eroarea la stratul de ieșire:

$$\delta_r^d = (t_r^d - o_r^d) \quad \text{sau} \quad \delta_r^d = o_r^d(1 - o_r^d)(t_r^d - o_r^d), \quad r = 1, \dots, R$$

Eroarea la stratul ascuns (propagată înapoi):

$$\delta_h^d = o_h^d(1 - o_h^d) \sum_{r=1}^R w_{hr}^2 \delta_r^d$$

Aceasta distribuie eroarea pe toate conexiunile, **proporțional cu ponderile**.

7.5 Actualizarea ponderilor

Între stratul ascuns și ieșire:

$$w_{hr}^2 = w_{hr}^2 + \eta \cdot \delta_r^d \cdot o_h^d, \quad h = 1, \dots, H; \quad r = 1, \dots, R$$

Între stratul de intrare și ascuns:

$$w_{ih}^1 = w_{ih}^1 + \eta \cdot \delta_h^d \cdot x_i^d, \quad i = 1, \dots, m; \quad h = 1, \dots, H$$

Unde η = rata de învățare.

7.6 Chain Rule (Regula lanțului)

Backpropagation folosește regula lanțului din calcul diferențial:

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Unde:

- $\frac{\partial E}{\partial \hat{y}}$ = cum afectează schimbarea ieșirii eroarea
- $\frac{\partial \hat{y}}{\partial z}$ = derivata funcției de activare
- $\frac{\partial z}{\partial w_j} = x_j$ = intrarea corespunzătoare ponderii

Intuiție: E ca o serie de roți conectate. Rotiți ultima, și efectul se transmite înapoi prin toate. Chain rule spune cât de mult trebuie rotită fiecare roată.

7.7 Condiții de oprire Backpropagation

- Eroarea a ajuns la 0 (sau sub un prag mic)
- S-a atins un număr maxim de iterații
- O iterație = procesarea unui singur exemplu; n iterații = o **epocă**

8. Proiectarea unei RNA

8.1 Câte straturi ascunse?

- **1 strat** — suficient pentru probleme simple (teorema aproximării universale)
- **2–3 straturi** — majoritatea problemelor nelineare
- **>3 straturi** — deep learning (imagini, text, audio)

8.2 Câți neuroni pe strat?

- Se aleg prin **experimentare** (cross-validare)
- Reguli empirice: între dimensiunea inputului și outputului, tipic 10–100

8.3 Inițializarea ponderilor

- Se folosesc **distribuții normale sau uniforme** cu medie zero
- Scopul: **evitarea simetriei** (dacă toți neuronii pornesc identic, învață același lucru)

8.4 Hiperparametrii

- **Learning rate** η : 0.001 – 0.1, ales empiric
 - **Regularizare** λ : controlează overfitting-ul
 - **Mini-batch size**: 32, 64 etc.
-

9. Probleme Comune și Soluții

9.1 Overfitting

Model se potrivește prea bine datelor de antrenament.

Soluții:

- **Regularizare (L1/L2)** — penalizează ponderile mari
- **Dropout** — oprirea aleatorie a unor neuroni în timpul antrenării
- **Early stopping** — oprirea antrenării când eroarea pe validare crește

9.2 Vanishing gradient

Gradienții devin extrem de mici în straturile profunde → nu se mai învață.

Soluții: folosirea ReLU în loc de sigmoid, normalizare pe batch.

10. CNN — Rețele Neuronale Convoluționale

10.1 Ce sunt?

Tip special de RNA creat pentru **prelucrarea datelor cu structură spațială** (imagini). Spre deosebire de MLP, CNN-urile înțeleg **poziția și vecinătatea** elementelor.

10.2 De ce sunt eficiente?

1. **Localitate** — fiecare neuron „vede” doar o regiune mică
2. **Partajarea greutăților** — același filtru pe toată imaginea
3. **Reducerea dimensiunii** — prin pooling
4. **Extracție automată de caracteristici**

10.3 Straturile unei CNN

Strat convoluțional

Aplică un **filtru (kernel)** pe regiuni mici ale imaginii:

$$S(i, j) = \sum_{m=0}^{f-1} \sum_{n=0}^{f-1} I(i+m, j+n) \cdot K(m, n)$$

Unde:

- I = imaginea de intrare
- K = filtrul (kernel), matrice mică (ex: 3×3)
- f = dimensiunea filtrului
- (i, j) = poziția curentă

Rezultatul = **hartă de activare (feature map)**.

Exemplu numeric: Imagine:

```
1 2 0
0 1 3
2 1 1
```

Kernel (dectecție margini):

```
1  0
0 -1
```

Se glisează filtrul peste imagine, se înmulțesc element cu element și se însumează.

Hiperparametrii stratului convoluțional

Simbol	Semnificație	Exemplu
W	Dimensiunea intrării	6×6 pixeli
F	Dimensiunea filtrului	3×3
P	Padding (zero-padding)	0 sau 1
S	Stride (pas de deplasare)	1 sau 2
K	Numărul de filtre	32

Formula dimensiunii ieșirii

$$\text{dimensiune ieșire} = \frac{W - F + 2P}{S} + 1$$

Exemplu: $W = 6, F = 3, P = 0, S = 1$: $\frac{6-3+0}{1} + 1 = 4 \rightarrow$ ieșire 4×4

Strat de pooling

Reduce dimensiunea hărții de activare, păstrând informația esențială.

- **Max pooling** — selectează valoarea maximă din fiecare fereastră
- **Average pooling** — calculează media

Exemplu Max Pooling (2×2 , stride 2):

```

1 1 2 4      →      6 8
5 6 7 8        3 4
3 2 1 0
1 2 3 4

```

Rolul pooling-ului:

- Reduce nr. de parametri
- Toleranță la translații mici
- Previne overfitting

Flatten + Dense Layers

După convoluții și pooling, hărțile sunt „aplatizate” (reshape în vector) și trecute prin straturi complet conectate (dense), ca la un MLP clasic.

Strat de ieșire

- **Softmax** — pentru clasificare multclasă
- **Sigmoid** — pentru clasificare binară

10.4 Arhitectura tipică CNN

1. Intrare: imagine (ex: 28×28 pixeli)
2. Convoluție + ReLU
3. Pooling
4. (Repetare convoluție + pooling de mai multe ori)
5. Flatten
6. Dense layers
7. Strat de ieșire (softmax)

11. Arhitecturi Speciale (menționate în curs)

Arhitectură	Utilizare
Perceptron simplu	Un singur strat, clasificare liniară
MLP (Multi-Layer Perceptron)	Unul sau mai multe straturi ascunse
CNN	Imagini
RNN (Recurrent Neural Networks)	Secvențe (text, vorbire)

12. Calculul Numărului de Coeficienți (CRITIC pentru examen!)

12.1 Formula generală

Între **2 straturi consecutive** cu a neuroni pe stratul din stânga și b neuroni pe stratul din dreapta:

$$\text{nr. ponderi (fără bias)} = a \times b$$

$$\text{nr. total (cu bias)} = a \times b + b$$

12.2 Exemplu concret — Subiect 4 din examen

"Câți coeficienți trebuie optimizați pentru o rețea neuronală cu un strat ascuns cu 5 neuroni, pentru predicția consumului de combustibil pe baza puterii motorului și a anului de fabricație? Exclueți bias-ul."

Datele:

- Strat de intrare: **2 neuroni** (putere motor, an fabricație)
- Strat ascuns: **5 neuroni**
- Strat de ieșire: **1 neuron** (consum combustibil — regresie)

Calcul (fără bias):

- Între intrare și ascuns: $2 \times 5 = 10$
- Între ascuns și ieșire: $5 \times 1 = 5$
- **Total:** $10 + 5 = 15$

Răspunsul la examen: b. 15

12.3 Exemplu cu bias inclus

Aceeași rețea, dar cu bias:

- Între intrare și ascuns: $2 \times 5 + 5 = 15$ (fiecare neuron ascuns are și un bias)
- Între ascuns și ieșire: $5 \times 1 + 1 = 6$
- **Total cu bias:** $15 + 6 = 21$

13. Avantaje și Dezavantaje RNA

Avantaje

- Pot învăța relații **complexe și nelineare**
- Foarte **flexibile** — se pot adapta la orice problemă
- Performanță excelentă pe imagini, text, audio

Dezavantaje

- Necesită **multe date** pentru antrenare
- **Cutie neagră** — dificil de interpretat ce a învățat
- Sensibile la **hiperparametri** (learning rate, arhitectură, nr. straturi)

- Pot cădea în **optime locale** (vs. SVM care oferă optimul global)

14. ⚠ Aspecte Critice pentru Examen

Tipare din SUBIECT_EXAMEN_2025

Subiect 1 — "O rețea neuronală convoluțională cu un singur strat, un singur neuron, 4 ponderi (3×3), un bias, input 6×6, stride 1. Care va fi dimensiunea feature map-ului rezultat?"

→ Formula: $\frac{W-F+2P}{S} + 1 = \frac{6-3+0}{1} + 1 = 4 \rightarrow \text{Feature map } 4 \times 4$

Subiect 4 — "Câți coeficienți trebuie optimizați? (fără bias)"

→ Nr. ponderi = intrări × ascunși + ascunși × ieșiri = $2 \times 5 + 5 \times 1 = 15$

Concepte-cheie de memorat

1. **Neuron**: $z = \sum w_i x_i + b$, apoi $y = f(z)$
2. **Weights** = contribuția intrărilor; **Bias** = deplasarea funcției
3. **Funcții de activare**: sigmoid (0, 1), ReLU $[0, \infty)$, tanh $(-1, 1)$, softmax (probabilități)
4. **Backpropagation**: forward pass → calcul eroare → backward pass (chain rule) → update ponderi
5. **Chain rule**: $\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$
6. **Perceptron**: poate AND, OR dar **NU** XOR
7. **Cross Entropy** → clasificare; **MSE** → regresie
8. **Softmax** → antrenare; **ArgMax** → predicție
9. **CNN: dimensiune ieșire** = $(W - F + 2P)/S + 1$
10. **Nr. coeficienți** = suma produselor (neuroni pe strat × neuroni pe stratul următor)
11. **Overfitting soluții**: regularizare, dropout, early stopping
12. **Epocă** = o trecere completă prin toate datele

Rezumat generat exclusiv din 04_ML_ANN.pdf și fișierele .md 4_1 → 4_9